

Proyecto SQL

Base de Datos Empresarial

Gestión de empleados, áreas y salarios mediante consultas y operaciones SQL

Autor	Christopher Alessandro Arias Diaz
Herramienta	DataGrip
Formato	SQLite (.sqlite)
Tipo	Base de Datos Empresarial Simulada
Área	Bases de Datos
Fecha	Abril 2026
Repositorio	GitHub

Índice de Contenido

1. Descripción General	4
Evidencia	4
2. Estructura de la Base de Datos	5
3. Desarrollo del Proyecto – Fases	5
Fase 1	5
Creación de la Tabla	5
Comando utilizado	5
Aprendizaje	5
Fase 2	5
Inserción de Datos	5
Comandos utilizados	5
Aprendizaje	6
Evidencia	6
Fase 3	7
Consulta de Datos (SELECT)	7
Comando utilizado	7
Aprendizaje	7
Evidencia	7
Fase 4	8
Filtrado de Datos (WHERE)	8
Comando utilizado	8
Aprendizaje	8
Evidencia	8
Fase 5	9
Búsqueda con LIKE	9
Comando utilizado	9
Aprendizaje	9
Evidencia	9
Fase 6	10
Conteo de Registros (COUNT)	10
Comando utilizado	10
Aprendizaje	10

Evidencia	10
Fase 7	11
Agrupación de Datos (GROUP BY)	11
Comando utilizado	11
Aprendizaje	11
Evidencia	11
Fase 8	12
Eliminación de Registros (DELETE)	12
Comando utilizado	12
Aprendizaje	12
Evidencia	12
Fase 9	13
Cálculo de Promedio (AVG)	13
Comandos utilizados	13
Aprendizaje	13
Evidencia	13
4. Pruebas Realizadas	14
5. Aprendizaje General	14
6. Dudas y Consideraciones	15
7. Conclusión	15

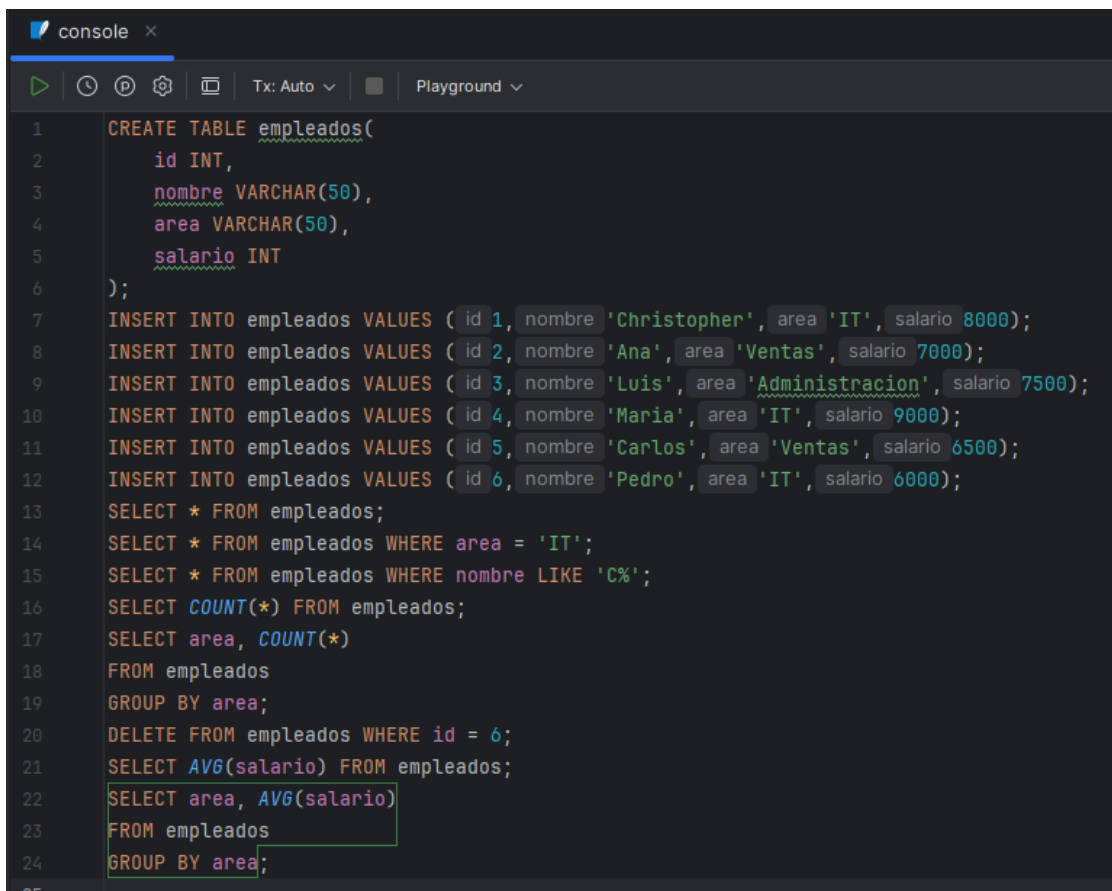
1. Descripción General

Se desarrolló una base de datos en SQL utilizando DataGrip con formato SQLite para simular un entorno empresarial. En este entorno se gestionan empleados, áreas de trabajo y salarios mediante diferentes consultas y operaciones SQL.

Herramienta DataGrip	Formato .sqlite	Tabla empleados	Registros 6 empleados
-------------------------	--------------------	--------------------	--------------------------

Evidencia

Vista general del playground en DataGrip con todos los comandos y resultados del proyecto.



```
1 CREATE TABLE empleados(
2     id INT,
3     nombre VARCHAR(50),
4     area VARCHAR(50),
5     salario INT
6 );
7 INSERT INTO empleados VALUES ( id 1, nombre 'Christopher', area 'IT', salario 8000);
8 INSERT INTO empleados VALUES ( id 2, nombre 'Ana', area 'Ventas', salario 7000);
9 INSERT INTO empleados VALUES ( id 3, nombre 'Luis', area 'Administracion', salario 7500);
10 INSERT INTO empleados VALUES ( id 4, nombre 'Maria', area 'IT', salario 9000);
11 INSERT INTO empleados VALUES ( id 5, nombre 'Carlos', area 'Ventas', salario 6500);
12 INSERT INTO empleados VALUES ( id 6, nombre 'Pedro', area 'IT', salario 6000);
13 SELECT * FROM empleados;
14 SELECT * FROM empleados WHERE area = 'IT';
15 SELECT * FROM empleados WHERE nombre LIKE 'C%';
16 SELECT COUNT(*) FROM empleados;
17 SELECT area, COUNT(*)
18 FROM empleados
19 GROUP BY area;
20 DELETE FROM empleados WHERE id = 6;
21 SELECT AVG(salario) FROM empleados;
22 SELECT area, AVG(salario)
23 FROM empleados
24 GROUP BY area;
```

2. Estructura de la Base de Datos

La base de datos contiene una tabla principal llamada empleados, compuesta por los siguientes campos:

Campo	Tipo	Descripción
id	INT	Identificador único del empleado
nombre	VARCHAR(50)	Nombre completo del empleado
area	VARCHAR(50)	Departamento o área de trabajo
salario	INT	Salario mensual del empleado

3. Desarrollo del Proyecto – Fases

Fase 1 Creación de la Tabla

Comando utilizado

```
CREATE TABLE empleados (  
    id INT,  
    nombre VARCHAR(50),  
    area VARCHAR(50),  
    salario INT  
);
```

Aprendizaje

Se comprendió cómo definir la estructura de una tabla en SQLite, estableciendo las columnas y sus tipos de datos. La instrucción CREATE TABLE permite declarar el esquema antes de insertar cualquier registro.

Fase 2 Inserción de Datos

Comandos utilizados

```
INSERT INTO empleados VALUES (1, 'Christopher', 'IT',  
8000);  
INSERT INTO empleados VALUES (2, 'Ana', 'Ventas',  
7000);  
INSERT INTO empleados VALUES (3, 'Luis', 'Administracion',  
7500);
```

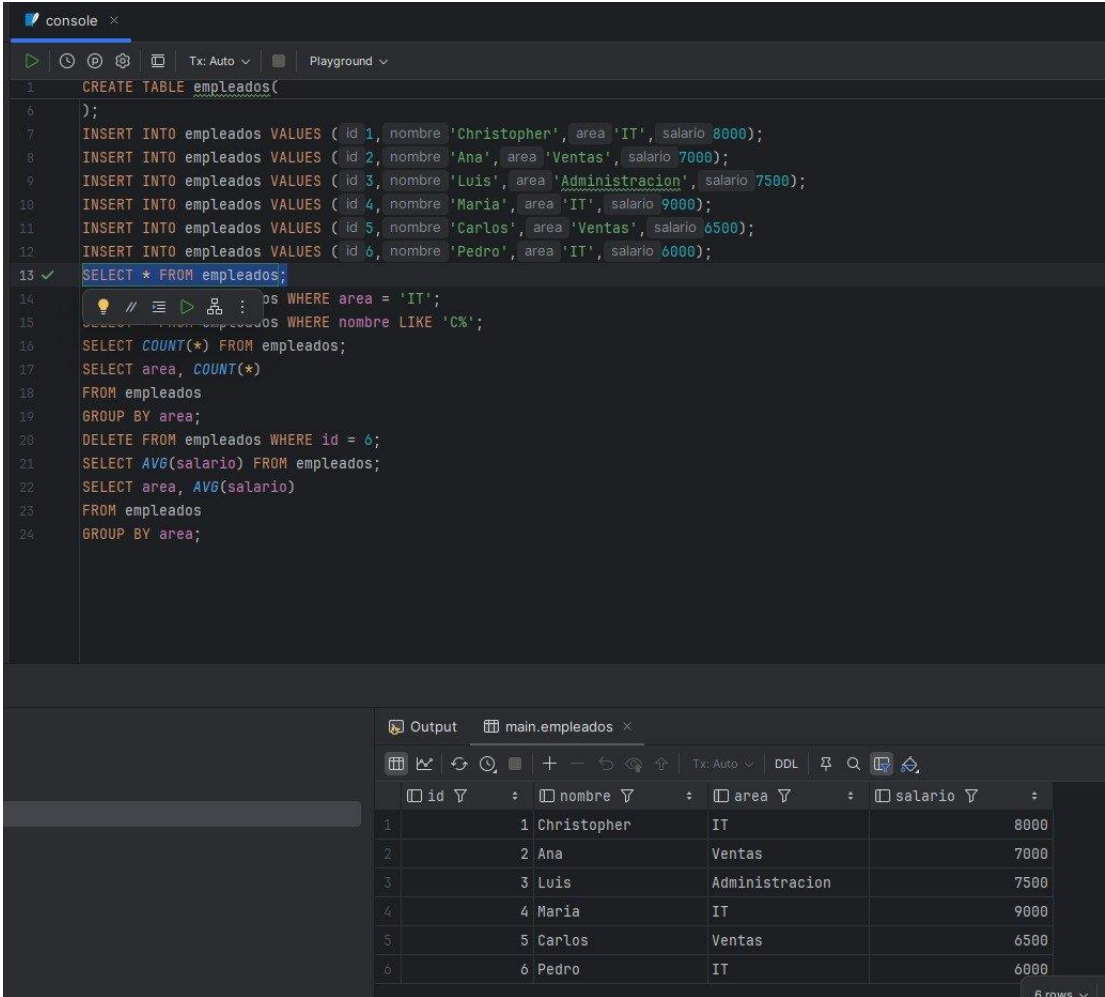
```
INSERT INTO empleados VALUES (4, 'Maria', 'IT', 9000);
INSERT INTO empleados VALUES (5, 'Carlos', 'Ventas', 6500);
INSERT INTO empleados VALUES (6, 'Pedro', 'IT', 6000);
```

Aprendizaje

Se aprendió a insertar registros usando INSERT INTO ... VALUES, representando información real de empleados de distintas áreas y rangos salariales.

Evidencia

Se visualizan los 6 empleados insertados en la tabla, incluyendo a Pedro (id 6) del área de IT.



```
1 CREATE TABLE empleados(
6 );
7 INSERT INTO empleados VALUES (id 1, nombre 'Christopher', area 'IT', salario 8000);
8 INSERT INTO empleados VALUES (id 2, nombre 'Ana', area 'Ventas', salario 7000);
9 INSERT INTO empleados VALUES (id 3, nombre 'Luis', area 'Administracion', salario 7500);
10 INSERT INTO empleados VALUES (id 4, nombre 'Maria', area 'IT', salario 9000);
11 INSERT INTO empleados VALUES (id 5, nombre 'Carlos', area 'Ventas', salario 6500);
12 INSERT INTO empleados VALUES (id 6, nombre 'Pedro', area 'IT', salario 6000);
13 SELECT * FROM empleados;
```

Output

id	nombre	area	salario
1	Christopher	IT	8000
2	Ana	Ventas	7000
3	Luis	Administracion	7500
4	Maria	IT	9000
5	Carlos	Ventas	6500
6	Pedro	IT	6000

Fase 3**Consulta de Datos (SELECT)****Comando utilizado**

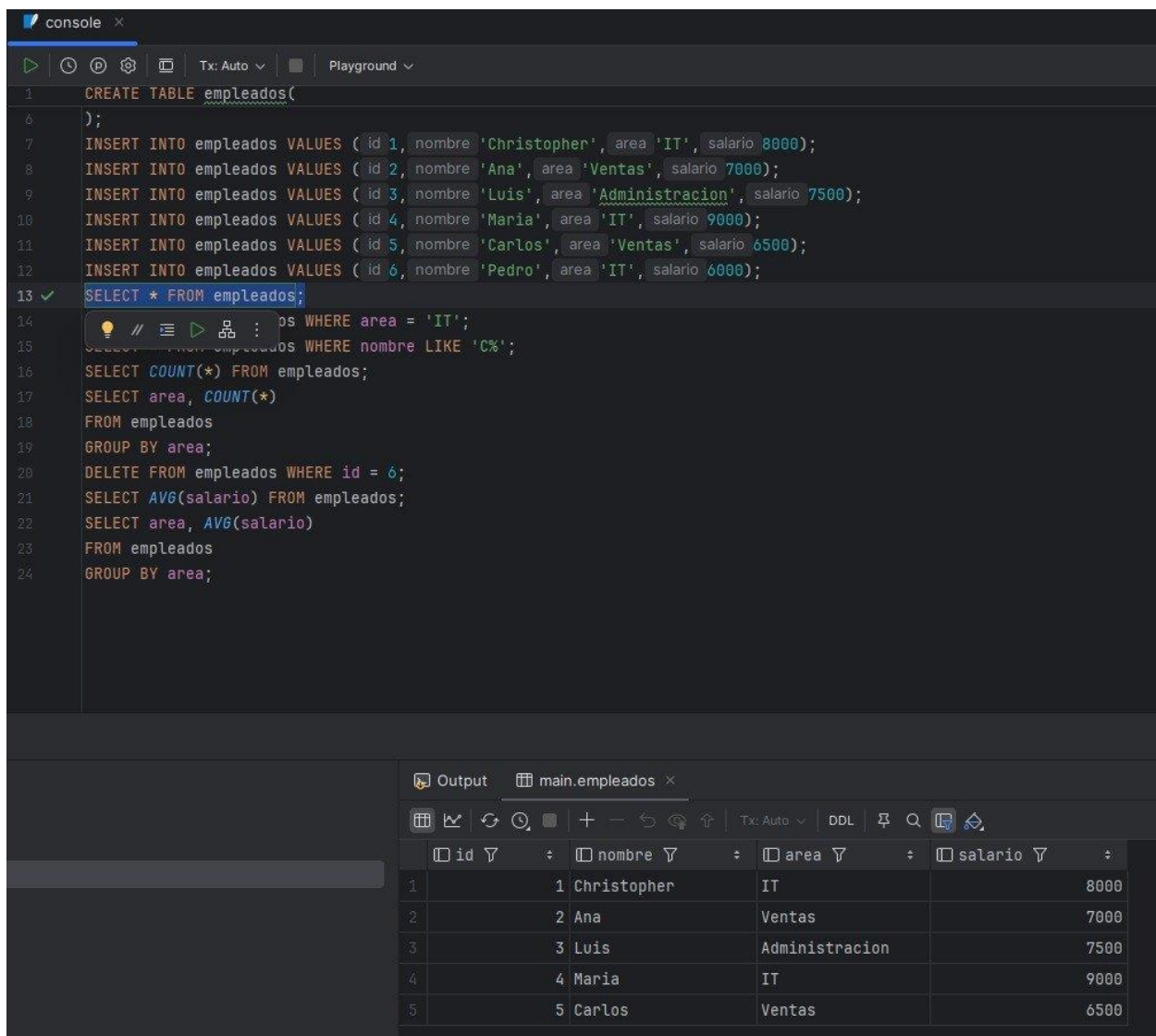
```
| SELECT * FROM empleados;
```

Aprendizaje

Se comprendió cómo visualizar todos los registros almacenados en la base de datos. El asterisco (*) indica que se recuperan todas las columnas disponibles.

Evidencia

*Resultado del SELECT * mostrando todos los empleados con sus respectivas áreas y salarios.*



```
1 CREATE TABLE empleados(
6 );
7 INSERT INTO empleados VALUES ( id 1, nombre 'Christopher', area 'IT', salario 8000);
8 INSERT INTO empleados VALUES ( id 2, nombre 'Ana', area 'Ventas', salario 7000);
9 INSERT INTO empleados VALUES ( id 3, nombre 'Luis', area 'Administracion', salario 7500);
10 INSERT INTO empleados VALUES ( id 4, nombre 'Maria', area 'IT', salario 9000);
11 INSERT INTO empleados VALUES ( id 5, nombre 'Carlos', area 'Ventas', salario 6500);
12 INSERT INTO empleados VALUES ( id 6, nombre 'Pedro', area 'IT', salario 6000);
13 ✓ SELECT * FROM empleados;
```

Output

id	nombre	area	salario
1	1 Christopher	IT	8000
2	2 Ana	Ventas	7000
3	3 Luis	Administracion	7500
4	4 Maria	IT	9000
5	5 Carlos	Ventas	6500

Fase 4 Filtrado de Datos (WHERE)

Comando utilizado

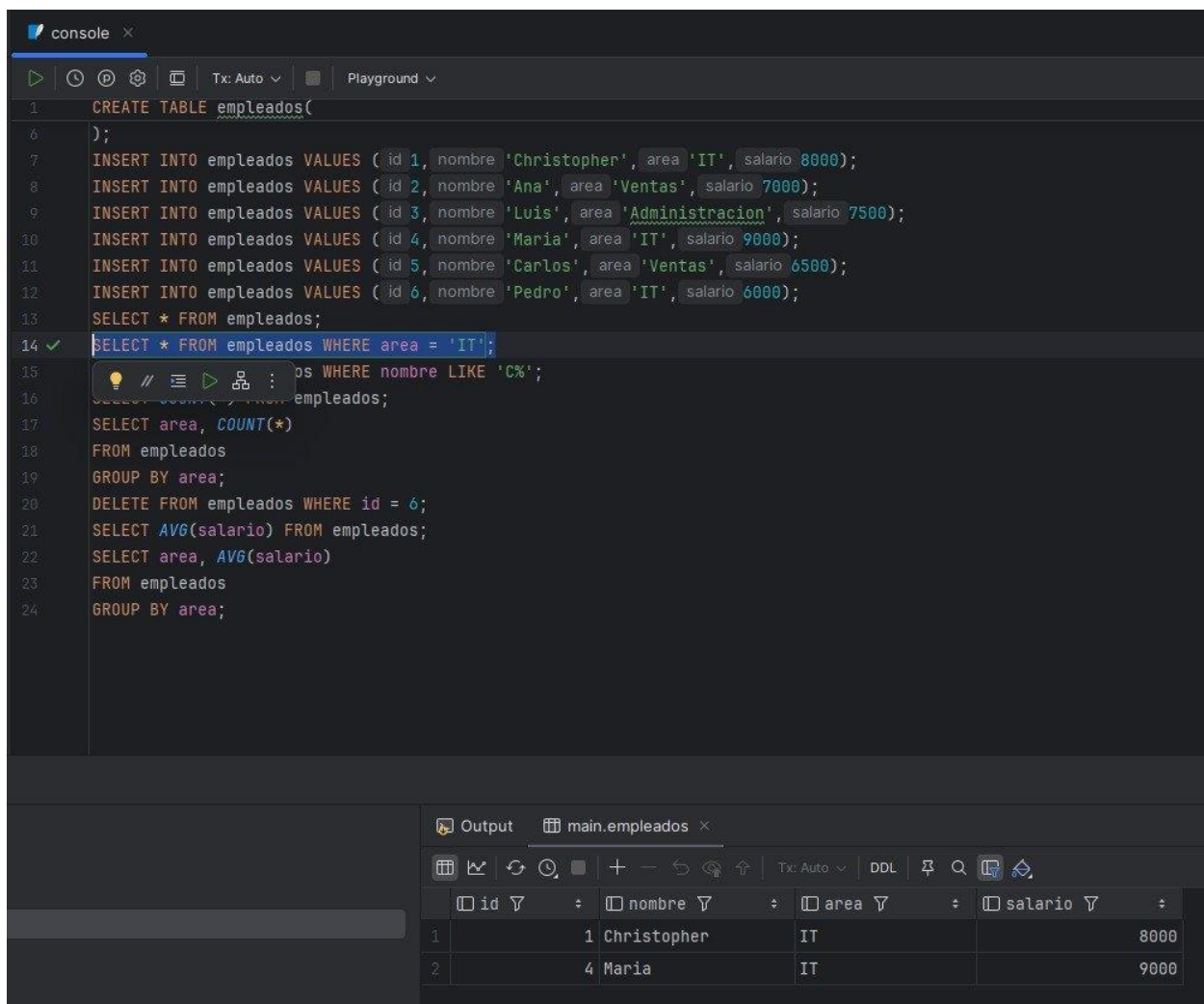
```
SELECT * FROM empleados WHERE area = 'IT';
```

Aprendizaje

Se aprendió a filtrar información específica mediante condiciones con la cláusula WHERE. En este caso se recuperaron únicamente los empleados pertenecientes al área de IT.

Evidencia

Resultado mostrando Christopher y Maria como empleados del área IT.



The screenshot shows a SQL IDE interface. The console window displays the following SQL commands:

```
1 CREATE TABLE empleados(  
6 );  
7 INSERT INTO empleados VALUES (id 1, nombre 'Christopher', area 'IT', salario 8000);  
8 INSERT INTO empleados VALUES (id 2, nombre 'Ana', area 'Ventas', salario 7000);  
9 INSERT INTO empleados VALUES (id 3, nombre 'Luis', area 'Administracion', salario 7500);  
10 INSERT INTO empleados VALUES (id 4, nombre 'Maria', area 'IT', salario 9000);  
11 INSERT INTO empleados VALUES (id 5, nombre 'Carlos', area 'Ventas', salario 6500);  
12 INSERT INTO empleados VALUES (id 6, nombre 'Pedro', area 'IT', salario 6000);  
13 SELECT * FROM empleados;  
14 ✓ SELECT * FROM empleados WHERE area = 'IT';  
15  
16  
17 SELECT area, COUNT(*)  
18 FROM empleados  
19 GROUP BY area;  
20 DELETE FROM empleados WHERE id = 6;  
21 SELECT AVG(salario) FROM empleados;  
22 SELECT area, AVG(salario)  
23 FROM empleados  
24 GROUP BY area;
```

The output window, titled "main.empleados", shows the results of the query executed in line 14. It displays a table with the following data:

id	nombre	area	salario
1	Christopher	IT	8000
4	Maria	IT	9000

Fase 5

Búsqueda con LIKE

Comando utilizado

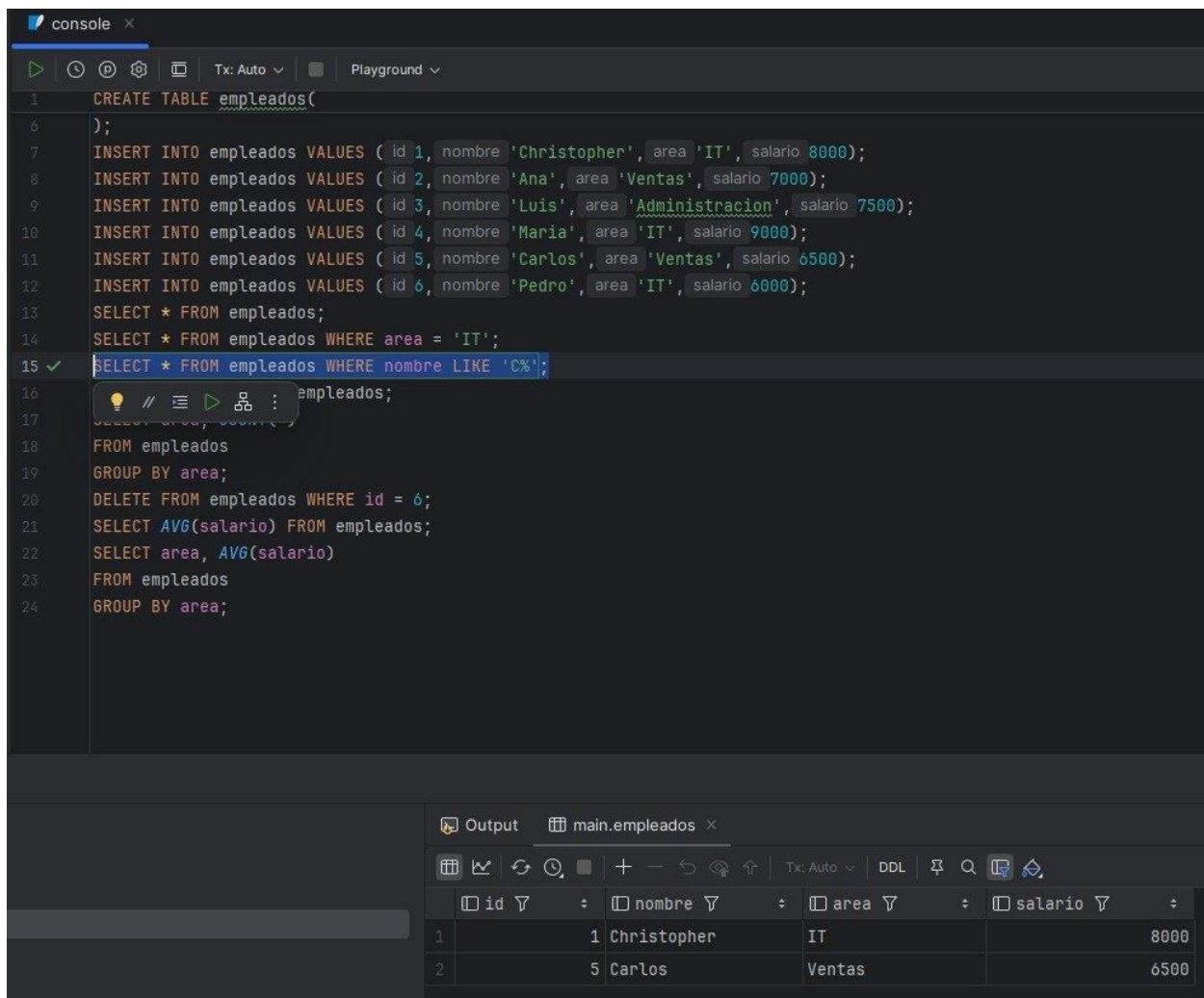
```
SELECT * FROM empleados WHERE nombre LIKE 'C%';
```

Aprendizaje

Se comprendió cómo buscar registros utilizando patrones con el operador LIKE. El patrón 'C%' recupera todos los empleados cuyo nombre comienza con la letra C.

Evidencia

El resultado muestra a Christopher (IT) y Carlos (Ventas), únicos empleados con nombre iniciando en 'C'.



The screenshot shows a SQL IDE with a script editor and an output window. The script defines a table 'empleados' with columns 'id', 'nombre', 'area', and 'salario'. It inserts six records: Christopher (IT, 8000), Ana (Ventas, 7000), Luis (Administracion, 7500), Maria (IT, 9000), Carlos (Ventas, 6500), and Pedro (IT, 6000). The script then executes a query: `SELECT * FROM empleados WHERE nombre LIKE 'C%';`. The output window displays the results of this query, showing two records: Christopher (IT, 8000) and Carlos (Ventas, 6500).

```
1 CREATE TABLE empleados(  
6 );  
7 INSERT INTO empleados VALUES (id 1, nombre 'Christopher', area 'IT', salario 8000);  
8 INSERT INTO empleados VALUES (id 2, nombre 'Ana', area 'Ventas', salario 7000);  
9 INSERT INTO empleados VALUES (id 3, nombre 'Luis', area 'Administracion', salario 7500);  
10 INSERT INTO empleados VALUES (id 4, nombre 'Maria', area 'IT', salario 9000);  
11 INSERT INTO empleados VALUES (id 5, nombre 'Carlos', area 'Ventas', salario 6500);  
12 INSERT INTO empleados VALUES (id 6, nombre 'Pedro', area 'IT', salario 6000);  
13 SELECT * FROM empleados;  
14 SELECT * FROM empleados WHERE area = 'IT';  
15 ✓ SELECT * FROM empleados WHERE nombre LIKE 'C%';  
16  
17  
18 FROM empleados  
19 GROUP BY area;  
20 DELETE FROM empleados WHERE id = 6;  
21 SELECT AVG(salario) FROM empleados;  
22 SELECT area, AVG(salario)  
23 FROM empleados  
24 GROUP BY area;
```

id	nombre	area	salario
1	Christopher	IT	8000
5	Carlos	Ventas	6500

Fase 6**Conteo de Registros (COUNT)****Comando utilizado**

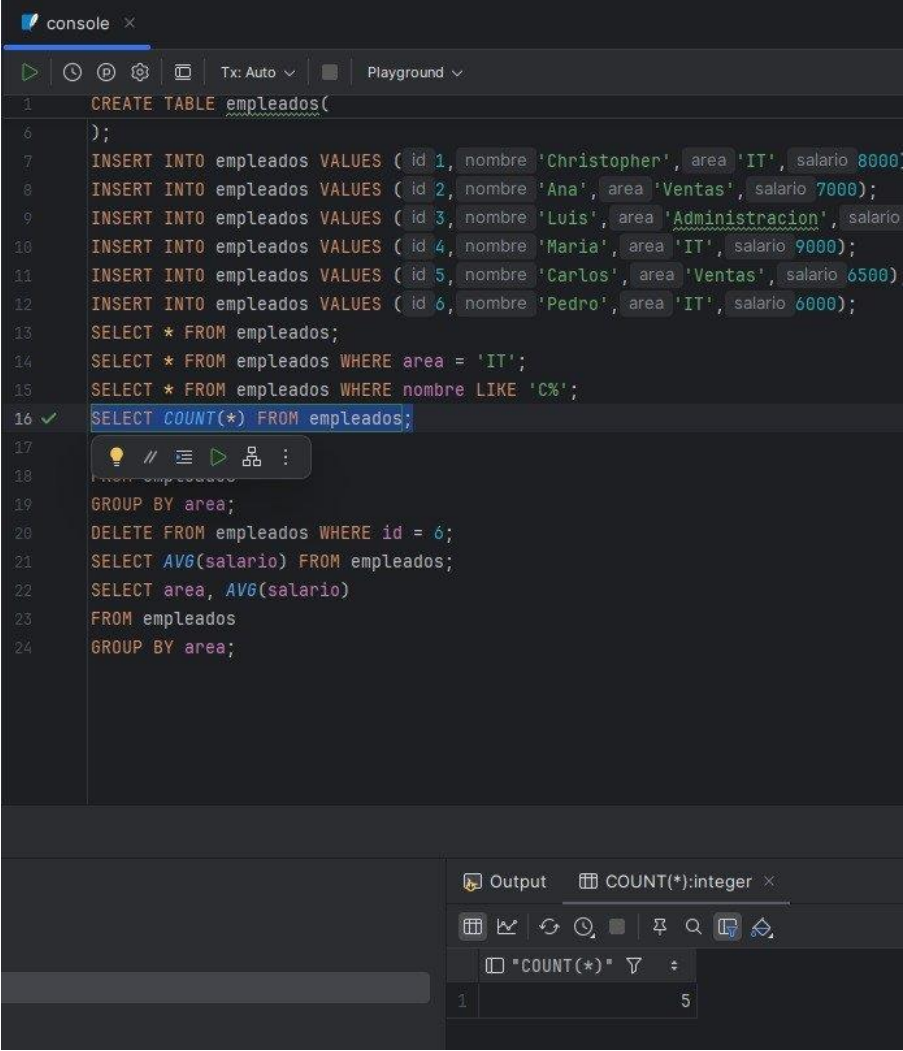
```
| SELECT COUNT(*) FROM empleados;
```

Aprendizaje

Se aprendió a contar la cantidad total de registros en una tabla mediante la función de agregación COUNT (*). El resultado devuelto fue 5 tras eliminar el registro con id 6 en la fase siguiente.

Evidencia

El COUNT () devuelve 5 registros.*



```
1 CREATE TABLE empleados(
6 );
7 INSERT INTO empleados VALUES ( id 1, nombre 'Christopher', area 'IT', salario 8000);
8 INSERT INTO empleados VALUES ( id 2, nombre 'Ana', area 'Ventas', salario 7000);
9 INSERT INTO empleados VALUES ( id 3, nombre 'Luis', area 'Administracion', salario
10 INSERT INTO empleados VALUES ( id 4, nombre 'Maria', area 'IT', salario 9000);
11 INSERT INTO empleados VALUES ( id 5, nombre 'Carlos', area 'Ventas', salario 6500);
12 INSERT INTO empleados VALUES ( id 6, nombre 'Pedro', area 'IT', salario 6000);
13 SELECT * FROM empleados;
14 SELECT * FROM empleados WHERE area = 'IT';
15 SELECT * FROM empleados WHERE nombre LIKE 'C%';
16 ✓ SELECT COUNT(*) FROM empleados;
17
18
19 GROUP BY area;
20 DELETE FROM empleados WHERE id = 6;
21 SELECT AVG(salario) FROM empleados;
22 SELECT area, AVG(salario)
23 FROM empleados
24 GROUP BY area;
```

Output COUNT(*) : integer x

"COUNT(*)"	
1	5

Fase 7

Agrupación de Datos (GROUP BY)

Comando utilizado

```
SELECT area, COUNT(*) FROM empleados GROUP BY area;
```

Aprendizaje

Se comprendió cómo agrupar datos por categorías con GROUP BY y aplicar funciones de agregación sobre cada grupo. El resultado muestra el número de empleados por área.

Evidencia

El resultado agrupa los empleados: Administración (1), IT (2), Ventas (2).

The screenshot shows a SQL console interface with a dark theme. The code in the editor includes creating a table named 'empleados', inserting six records with various attributes like id, nombre, area, and salario, and then executing a query to select area and count(*) grouped by area. The result of the query is displayed in a table at the bottom right.

```
1 CREATE TABLE empleados(
6 );
7 INSERT INTO empleados VALUES (id 1, nombre 'Christopher', area 'IT', salario 8000);
8 INSERT INTO empleados VALUES (id 2, nombre 'Ana', area 'Ventas', salario 7000);
9 INSERT INTO empleados VALUES (id 3, nombre 'Luis', area 'Administracion', salario 7500);
10 INSERT INTO empleados VALUES (id 4, nombre 'Maria', area 'IT', salario 9000);
11 INSERT INTO empleados VALUES (id 5, nombre 'Carlos', area 'Ventas', salario 6500);
12 INSERT INTO empleados VALUES (id 6, nombre 'Pedro', area 'IT', salario 6000);
13 SELECT * FROM empleados;
14 SELECT * FROM empleados WHERE area = 'IT';
15 SELECT * FROM empleados WHERE nombre LIKE 'C%';
16 SELECT COUNT(*) FROM empleados;
17 ✓ SELECT area, COUNT(*)
18 FROM empleados
19 GROUP BY area;
20 WHERE id = 6;
21 FROM empleados;
22 SELECT area, AVG(salario)
23 FROM empleados
24 GROUP BY area;
```

area	"COUNT(*)"
Administracion	1
IT	2
Ventas	2

Fase 8 Eliminación de Registros (DELETE)

Comando utilizado

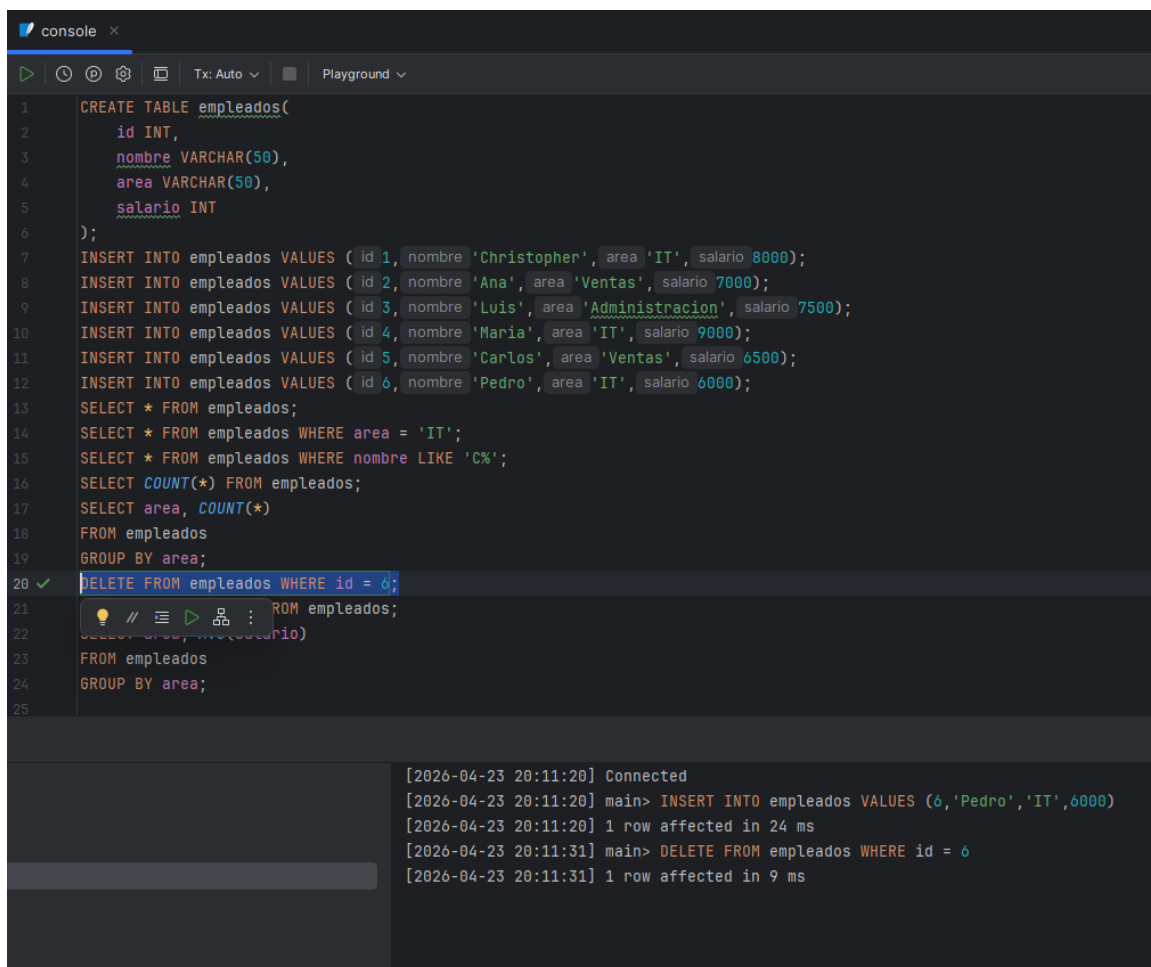
```
DELETE FROM empleados WHERE id = 6;
```

Aprendizaje

Se aprendió a eliminar registros específicos con DELETE FROM ... WHERE. Este comando eliminó el registro de Pedro (id 6) del área IT con salario 6000.

Evidencia

El resultado muestra la eliminación de un registro.



```
console x
Tx: Auto Playground
1 CREATE TABLE empleados(
2     id INT,
3     nombre VARCHAR(50),
4     area VARCHAR(50),
5     salario INT
6 );
7 INSERT INTO empleados VALUES (id 1, nombre 'Christopher', area 'IT', salario 8000);
8 INSERT INTO empleados VALUES (id 2, nombre 'Ana', area 'Ventas', salario 7000);
9 INSERT INTO empleados VALUES (id 3, nombre 'Luis', area 'Administracion', salario 7500);
10 INSERT INTO empleados VALUES (id 4, nombre 'Maria', area 'IT', salario 9000);
11 INSERT INTO empleados VALUES (id 5, nombre 'Carlos', area 'Ventas', salario 6500);
12 INSERT INTO empleados VALUES (id 6, nombre 'Pedro', area 'IT', salario 6000);
13 SELECT * FROM empleados;
14 SELECT * FROM empleados WHERE area = 'IT';
15 SELECT * FROM empleados WHERE nombre LIKE 'C%';
16 SELECT COUNT(*) FROM empleados;
17 SELECT area, COUNT(*)
18 FROM empleados
19 GROUP BY area;
20 ✓ DELETE FROM empleados WHERE id = 6;
21 FROM empleados;
22 FROM empleados
23 GROUP BY area;
25
[2026-04-23 20:11:20] Connected
[2026-04-23 20:11:20] main> INSERT INTO empleados VALUES (6,'Pedro','IT',6000)
[2026-04-23 20:11:20] 1 row affected in 24 ms
[2026-04-23 20:11:31] main> DELETE FROM empleados WHERE id = 6
[2026-04-23 20:11:31] 1 row affected in 9 ms
```

Fase 9

Cálculo de Promedio (AVG)

Comandos utilizados

```
SELECT AVG(salario) FROM empleados;

-- Promedio por área:
SELECT area, AVG(salario) FROM empleados GROUP BY area;
```

Aprendizaje

Se comprendió cómo realizar cálculos estadísticos sobre columnas numéricas con AVG. También se combinó con GROUP BY para obtener el salario promedio por departamento.

Evidencia

El resultado muestra el promedio de acuerdo al salario de todos los empleados.

```
1 CREATE TABLE empleados(
2   id INT,
3   nombre VARCHAR(50),
4   area VARCHAR(50),
5   salario INT
6 );
7 INSERT INTO empleados VALUES (id 1, nombre 'Christopher', area 'IT', salario 8000);
8 INSERT INTO empleados VALUES (id 2, nombre 'Ana', area 'Ventas', salario 7000);
9 INSERT INTO empleados VALUES (id 3, nombre 'Luis', area 'Administracion', salario 7500);
10 INSERT INTO empleados VALUES (id 4, nombre 'Maria', area 'IT', salario 9000);
11 INSERT INTO empleados VALUES (id 5, nombre 'Carlos', area 'Ventas', salario 6500);
12 INSERT INTO empleados VALUES (id 6, nombre 'Pedro', area 'IT', salario 6000);
13 SELECT * FROM empleados;
14 SELECT * FROM empleados WHERE area = 'IT';
15 SELECT * FROM empleados WHERE nombre LIKE 'C%';
16 SELECT COUNT(*) FROM empleados;
17 SELECT area, COUNT(*)
18 FROM empleados
19 GROUP BY area;
20 DELETE FROM empleados WHERE id = 6;
21 ✓ SELECT AVG(salario) FROM empleados;
22 SELECT area, AVG(salario)
23 FROM empleados
24 GROUP BY area;
```

AVG(salario)
7600

```
1 CREATE TABLE empleados(
2   id INT,
3   nombre VARCHAR(50),
4   area VARCHAR(50),
5   salario INT
6 );
7 INSERT INTO empleados VALUES (id 1, nombre 'Christopher', area 'IT', salario 8000);
8 INSERT INTO empleados VALUES (id 2, nombre 'Ana', area 'Ventas', salario 7000);
9 INSERT INTO empleados VALUES (id 3, nombre 'Luis', area 'Administracion', salario 7500);
10 INSERT INTO empleados VALUES (id 4, nombre 'Maria', area 'IT', salario 9000);
11 INSERT INTO empleados VALUES (id 5, nombre 'Carlos', area 'Ventas', salario 6500);
12 INSERT INTO empleados VALUES (id 6, nombre 'Pedro', area 'IT', salario 6000);
13 SELECT * FROM empleados;
14 SELECT * FROM empleados WHERE area = 'IT';
15 SELECT * FROM empleados WHERE nombre LIKE 'C%';
16 SELECT COUNT(*) FROM empleados;
17 SELECT area, COUNT(*)
18 FROM empleados
19 GROUP BY area;
20 DELETE FROM empleados WHERE id = 6;
21 SELECT AVG(salario) FROM empleados;
22 ✓ SELECT area, AVG(salario)
23 FROM empleados
24 GROUP BY area;
```

area	AVG(salario)
Administracion	7500
IT	8500
Ventas	6750

4. Pruebas Realizadas

Prueba	Comando	Resultado
Visualización de todos los registros	SELECT *	✓ Exitoso
Filtrado por área (IT)	WHERE	✓ Exitoso
Búsqueda por patrón de nombre	LIKE	✓ Exitoso
Conteo total de registros	COUNT(*)	✓ Exitoso
Agrupación por área	GROUP BY	✓ Exitoso
Eliminación de registro por ID	DELETE	✓ Exitoso
Cálculo de salario promedio	AVG	✓ Exitoso
Promedio de salario por área	AVG + GROUP BY	✓ Exitoso

5. Aprendizaje General

Conocimiento adquirido	Aplicación práctica
Creación de tablas (CREATE TABLE)	Definir esquema empresarial
Inserción de registros (INSERT INTO)	Poblar tabla con empleados reales
Consultas completas (SELECT *)	Visualizar todos los datos
Filtrado de datos (WHERE / LIKE)	Búsquedas por área y nombre
Funciones de agregación (COUNT / AVG)	Análisis estadístico de salarios
Agrupación de resultados (GROUP BY)	Resumen por departamento
Eliminación de registros (DELETE)	Control del ciclo de vida de datos

6. Dudas y Consideraciones

- Se presentaron dudas iniciales en la sintaxis de algunos comandos, las cuales fueron resueltas mediante la práctica iterativa y la consulta de documentación.
- Al trabajar con SQLite en DataGrip, se identificaron diferencias menores respecto a otros motores SQL como MySQL o PostgreSQL (por ejemplo, tipos de datos más flexibles en SQLite).
- La eliminación del registro con id = 6 (Pedro) fue intencional para demostrar el funcionamiento del comando DELETE y su efecto en operaciones de conteo posteriores.

7. Conclusión

Este proyecto permitió comprender el uso de SQL en un entorno práctico utilizando DataGrip con SQLite, aplicando consultas, filtros y análisis de datos que simulan un escenario real de gestión empresarial.

Los conocimientos adquiridos abarcan desde la creación y estructura de la base de datos hasta la manipulación avanzada de registros mediante funciones de agregación, lo que representa una base sólida para el manejo de datos en entornos profesionales.